

Date:

Exp No:5

Aim:implement producer consumer problem and solve the problem using semaphores.

Theory

The **Producer–Consumer Problem** is a classic synchronization problem in **Operating Systems**. It describes a situation where:

- A **Producer** process produces data (items)
- A **Consumer** process consumes the data
- Both share a common **buffer** of fixed size

The main challenge is to ensure **proper synchronization** so that:

- Producer does not insert data when the buffer is **full**
- Consumer does not remove data when the buffer is **empty**
- Both do not access the buffer **simultaneously** (avoid race condition)

Consider:

- A fixed-size buffer of size N
- One or more producers
- One or more consumers

Operations:

- Producer → adds item to buffer
- Consumer → removes item from buffer

Critical Section

The buffer is a shared resource, so it becomes a **critical section**. Proper synchronization is required to avoid inconsistency.

1. Write the c program OS_exp5.c

```
#include <stdio.h>
#include <stdlib.h>
```

```
int mutex = 1;
int full = 0;
int empty;    // Buffer size will be taken from user
int x = 0;
```

```
int wait(int s)
{
    return (--s);
}
```

```

int signal(int s)
{
    return (++s);
}

void producer()
{
    mutex = wait(mutex);
    empty = wait(empty);
    full = signal(full);
    x++;

    printf("\nProducer produced item %d", x);

    mutex = signal(mutex);
}

void consumer()
{
    mutex = wait(mutex);
    full = wait(full);
    empty = signal(empty);

    printf("\nConsumer consumed item %d", x);
    x--;

    mutex = signal(mutex);
}

int main()
{
    int choice;

    printf("Enter Buffer Size: ");
    scanf("%d", &empty); // Taking buffer size from user

    while (1)
    {
        printf("\n\n1. Producer\n2. Consumer\n3. Exit");
        printf("\nEnter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                if ((mutex == 1) && (empty != 0))
                    producer();
                else
                    printf("\nBuffer is Full!");
                break;

```

```

    case 2:
        if ((mutex == 1) && (full != 0))
            consumer();
        else
            printf("\nBuffer is Empty!");
        break;

    case 3:
        exit(0);

    default:
        printf("\nInvalid Choice!");
}

return 0;
}

```

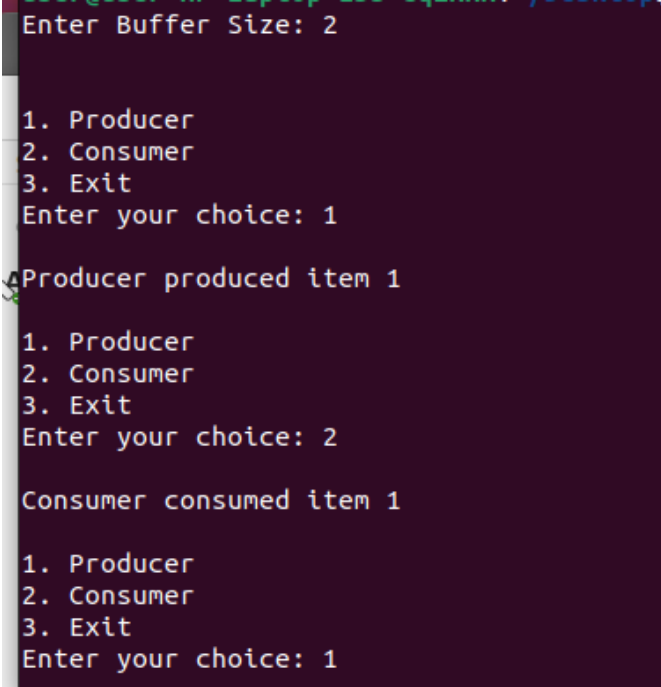
2.Compile and Run

```

gcc OS_exp4_1.c
./a.out

```

3.Expected Output



```

Enter Buffer Size: 2

1. Producer
2. Consumer
3. Exit
Enter your choice: 1

Producer produced item 1

1. Producer
2. Consumer
3. Exit
Enter your choice: 2

Consumer consumed item 1

1. Producer
2. Consumer
3. Exit
Enter your choice: 1

```

Result

The Producer–Consumer problem was successfully implemented using semaphore concepts